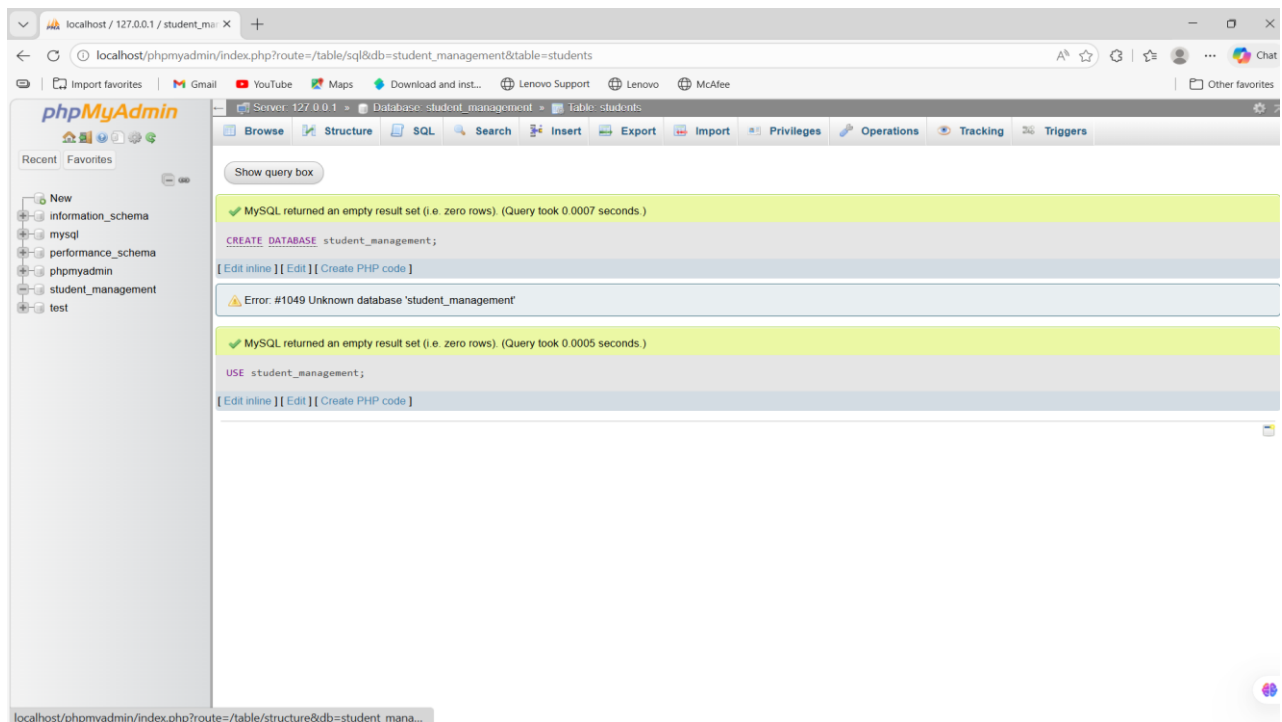


MySQL Professional Student Assessment

Database Design

Create a database named student_management

```
CREATE DATABASE student_management;  
USE student_management;
```



1. Students Table

```
CREATE TABLE Students (  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    age INT  
);
```

2. Courses Table

```
CREATE TABLE Courses (  
    course_id INT AUTO_INCREMENT PRIMARY KEY,  
    course_name VARCHAR(50) NOT NULL  
);
```

3. Enrollments Table

```
CREATE TABLE Enrollments (  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    course_id INT AUTO_INCREMENT PRIMARY KEY,  
    enrollment_date DATE NOT NULL,  
    enrollment_status VARCHAR(50) NOT NULL  
);
```

```

enroll_id INT AUTO_INCREMENT PRIMARY KEY,
student_id INT,
course_id INT,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

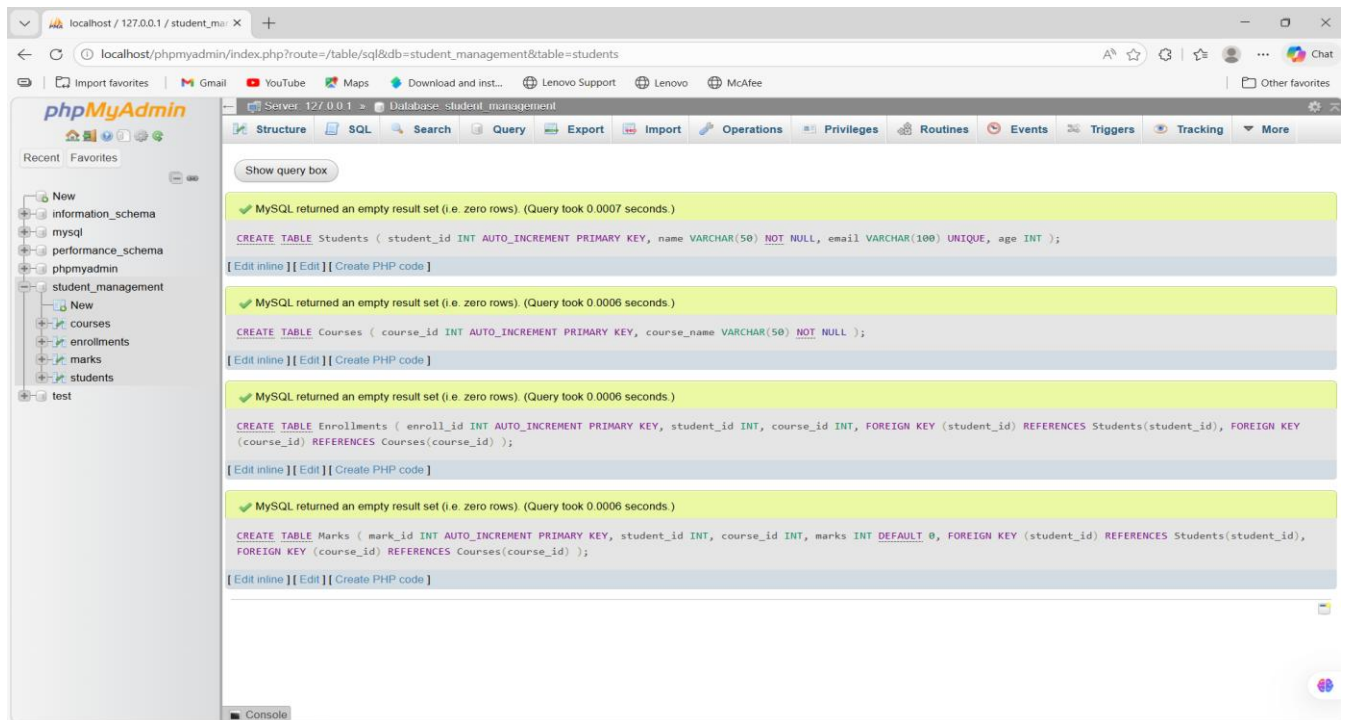
```

4. Marks Table

```

CREATE TABLE Marks (
mark_id INT AUTO_INCREMENT PRIMARY KEY,
student_id INT,
course_id INT,
marks INT DEFAULT 0,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

```



1. Students Table – Insert Query

```

INSERT INTO Students (name, email, age) VALUES
('chithra', 'chithra@gmail.com', 20),
('shree', 'shree@gmail.com', 21),
('divya', 'divya@gmail.com', 22),
('sony', 'sony@gmail.com', 19),
('deepa', 'deepa@gmail.com', 23),
('vinod', 'vinod@gmail.com', 20),
('chandu', 'chandu@gmail.com', 22),

```

```
('priya', 'priya@gmail.com', 21),  
( 'moulya', 'moulya@gmail.com', 20),  
( 'deeksha', 'deeksha@gmail.com', 19);
```

2. Courses Table – Insert Query

```
INSERT INTO Courses (course_name) VALUES  
( 'DBMS'),  
( 'Python'),  
( 'AI'),  
( 'ML'),  
( 'Cloud');
```

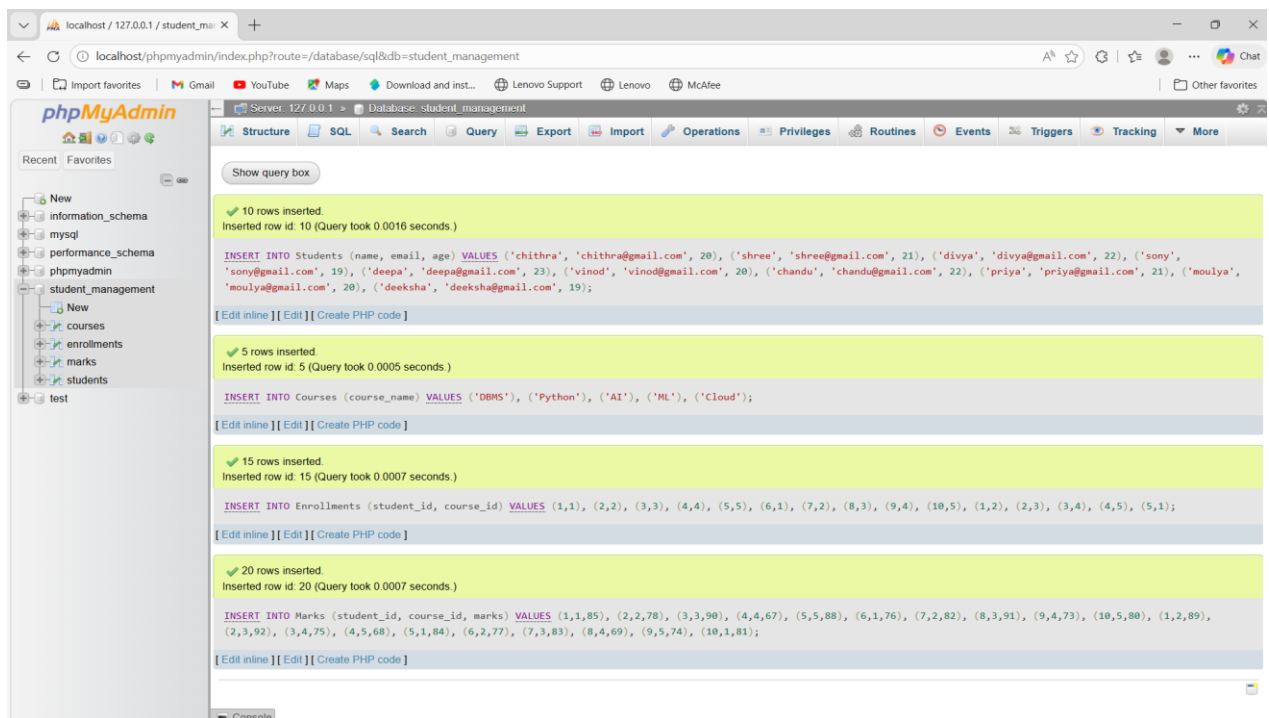
3. Enrollments Table – Insert Query

```
INSERT INTO Enrollments (student_id, course_id) VALUES  
(1,1),  
(2,2),  
(3,3),  
(4,4),  
(5,5),  
(6,1),  
(7,2),  
(8,3),  
(9,4),  
(10,5),  
(1,2),  
(2,3),  
(3,4),  
(4,5),  
(5,1);
```

4. Marks Table – Insert Query

```
INSERT INTO Marks (student_id, course_id, marks) VALUES  
(1,1,85),  
(2,2,78),  
(3,3,90),  
(4,4,67),  
(5,5,88),  
(6,1,76),  
(7,2,82),  
(8,3,91),  
(9,4,73),  
(10,5,80),  
(1,2,89),  
(2,3,92),
```

(3,4,75),
(4,5,68),
(5,1,84),
(6,2,77),
(7,3,83),
(8,4,69),
(9,5,74),
(10,1,81);



Part 3: SQL Queries

1 Basic queries: SELECT, WHERE, ORDER BY, LIMIT

- **SELECT**

SELECT * FROM Students WHERE age > 20;

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds)

```
SELECT * FROM Students WHERE age > 20;
```

Extra options

student_id	name	email	age
2	shree	shree@gmail.com	21
3	divya	divya@gmail.com	22
5	deepa	deepa@gmail.com	23
7	chandu	chandu@gmail.com	22
8	priya	priya@gmail.com	21

Query results operations: Print, Copy to clipboard, Export, Display chart, Create view

Bookmark this SQL query

- **ORDER BY**
SELECT * FROM students ORDER BY age DESC;

Showing rows 0 - 9 (10 total, Query took 0.0005 seconds) [age: 23... - 19...]

```
SELECT * FROM students ORDER BY age DESC;
```

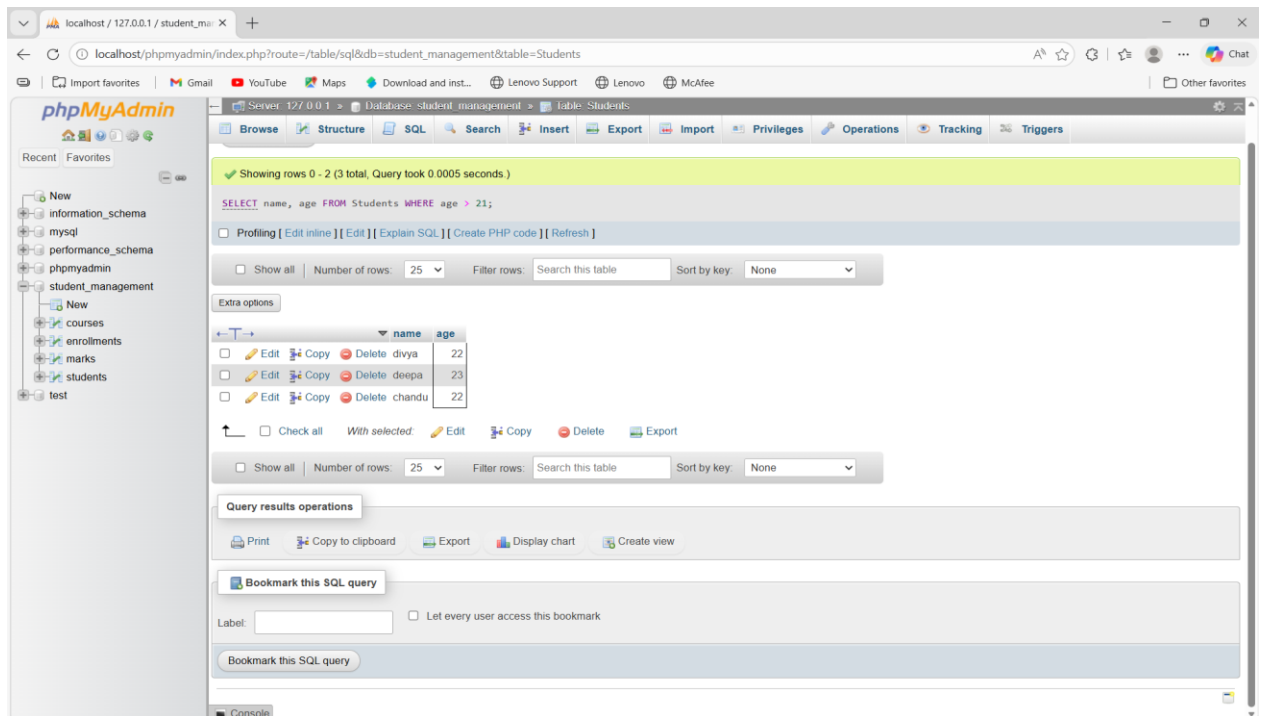
Extra options

student_id	name	email	age
5	deepa	deepa@gmail.com	23
3	divya	divya@gmail.com	22
7	chandu	chandu@gmail.com	22
2	shree	shree@gmail.com	21
8	priya	priya@gmail.com	21
1	chithra	chithra@gmail.com	20
9	moulya	moulya@gmail.com	20
6	vinod	vinod@gmail.com	20
4	sony	sony@gmail.com	19
10	deeksha	deeksha@gmail.com	19

Query results operations: Print, Copy to clipboard, Export, Display chart, Create view

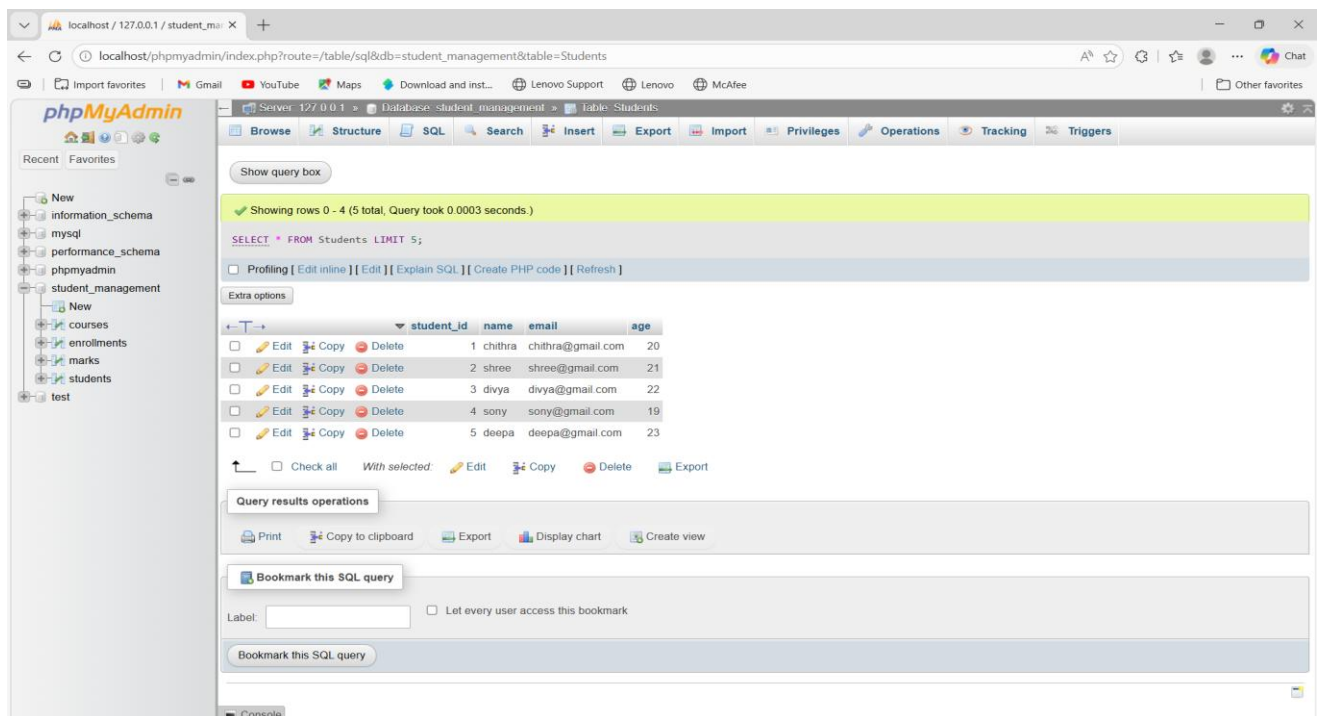
- **WHERE**

SELECT name, age FROM Students WHERE age > 21;



● LIMIT

`SELECT * FROM Students LIMIT 5;`



2 Logical operators: AND, OR, NOT

`SELECT * FROM Students`

`WHERE age > 20 AND age < 23;`

The screenshot shows the phpMyAdmin interface with the 'Students' table selected. The SQL query entered is: `SELECT * FROM Students WHERE age > 20 AND age < 23;`. The result shows 4 rows of student data.

student_id	name	email	age
2	shree	shree@gmail.com	21
3	divya	divya@gmail.com	22
7	chandu	chandu@gmail.com	22
8	priya	priya@gmail.com	21

OR OPERATOR

SELECT * FROM Students

WHERE age < 20 OR age > 22;

The screenshot shows the phpMyAdmin interface with the 'Students' table selected. The SQL query entered is: `SELECT * FROM Students WHERE age < 20 OR age > 22;`. The result shows 3 rows of student data.

student_id	name	email	age
4	sony	sony@gmail.com	19
5	deepa	deepa@gmail.com	23
10	deeksha	deeksha@gmail.com	19

NOT OPERATOR

SELECT * FROM Students

WHERE NOT age = 20;

The screenshot shows the phpMyAdmin interface with the 'Students' table selected. The SQL query executed is `SELECT * FROM Students WHERE NOT age = 20;`. The results show 6 rows of student data.

student_id	name	email	age
2	shree	shree@gmail.com	21
3	divya	divya@gmail.com	22
4	sony	sony@gmail.com	19
5	deepa	deepa@gmail.com	23
7	chandu	chandu@gmail.com	22
8	priya	priya@gmail.com	21

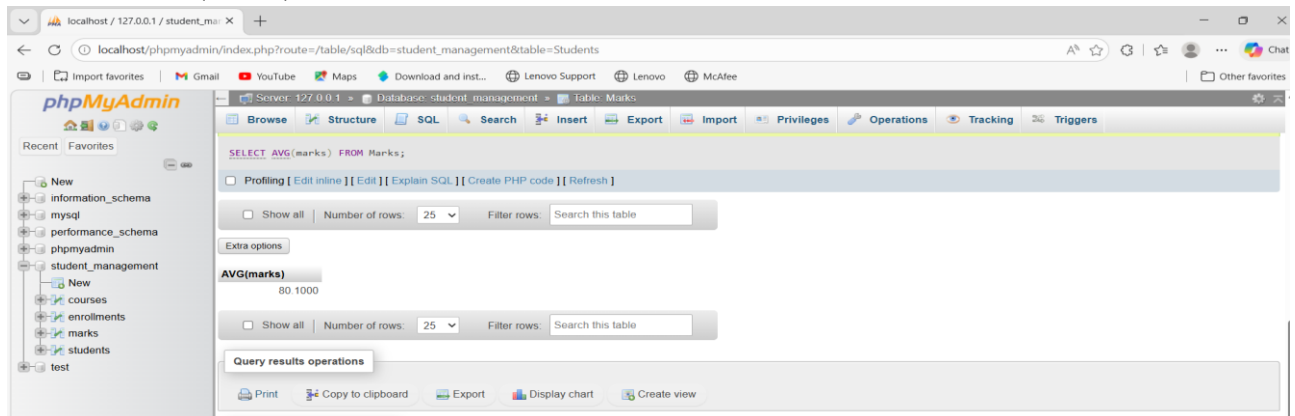
Aggregate functions: MIN, MAX, COUNT, AVG, SUM

SELECT COUNT(*) FROM Students;

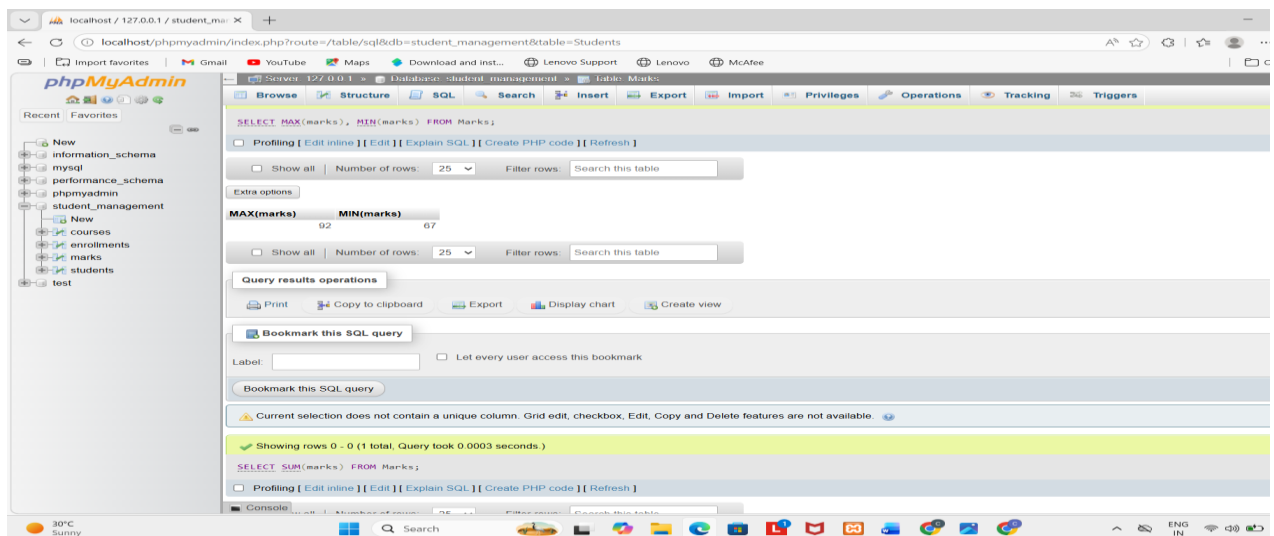
The screenshot shows the phpMyAdmin interface with the 'Marks' table selected. The SQL query executed is `SELECT COUNT(*) FROM Students;`. The result shows a single row with the count value 10.

COUNT(*)
10

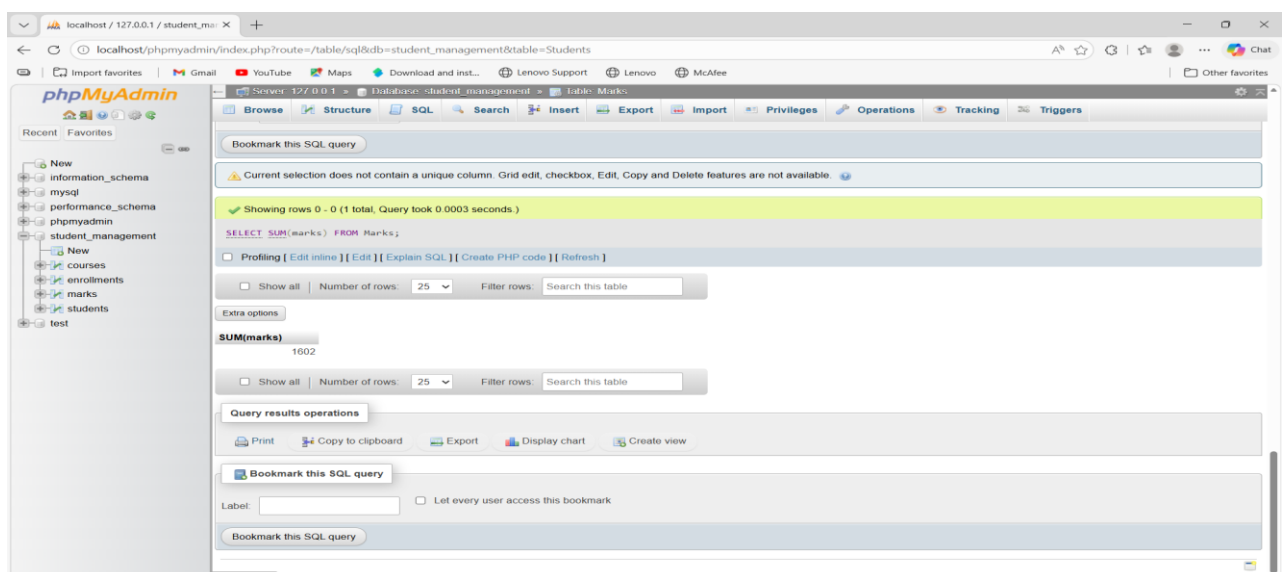
SELECT AVG(marks) FROM Marks;



SELECT MAX(marks), MIN(marks) FROM Marks;



SELECT SUM(marks) FROM Marks;



4 Filtering: LIKE, IN, BETWEEN

SELECT * FROM Students WHERE name LIKE 'A%';

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
SELECT * FROM Students WHERE name LIKE 'A%';
```

[Profiling](#) [Edit inline](#) [Edit](#) [Explain SQL](#) [Create PHP code](#) [Refresh](#)

student_id	name	email	age
------------	------	-------	-----

Query results operations

[Create view](#)

[Bookmark this SQL query](#)

Label: ☐ Let every user access this bookmark

[Bookmark this SQL query](#)

Showing rows 0 - 6 (7 total, Query took 0.0005 seconds.) [age: 22... - 20...]

SELECT * FROM Students WHERE age BETWEEN 20 AND 22;

localhost / 127.0.0.1 / student_ma: X

localhost/phpmyadmin/index.php?route=/table/sql&db=student_management&table=Marks

Server: 127.0.0.1 Database: student_management Table: Students

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Tracking](#) [Triggers](#)

```
SELECT * FROM Students WHERE age BETWEEN 20 AND 22 ORDER BY age DESC
```

[Profiling](#) [Edit inline](#) [Edit](#) [Explain SQL](#) [Create PHP code](#) [Refresh](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	student_id	name	email	age
☐	3	divya	divya@gmail.com	22
☐	7	chandu	chandu@gmail.com	22
☐	2	shree	shree@gmail.com	21
☐	8	priya	priya@gmail.com	21
☐	1	chithra	chithra@gmail.com	20
☐	6	vinod	vinod@gmail.com	20
☐	9	moulya	moulya@gmail.com	20

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

SELECT * FROM Students WHERE student_id IN (1,2,3);

localhost / 127.0.0.1 / student_ma: X

localhost/phpmyadmin/index.php?route=/table/sql&db=student_management&table=Marks

Server: 127.0.0.1 Database: student_management Table: Students

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Tracking](#) [Triggers](#)

```
SELECT * FROM Students WHERE student_id IN (1,2,3) ORDER BY age DESC
```

[Profiling](#) [Edit inline](#) [Edit](#) [Explain SQL](#) [Create PHP code](#) [Refresh](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	student_id	name	email	age
☐	3	divya	divya@gmail.com	22
☐	2	shree	shree@gmail.com	21
☐	1	chithra	chithra@gmail.com	20

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

Label: ☐ Let every user access this bookmark

[Bookmark this SQL query](#)

Console

5 Joins: INNER, LEFT, RIGHT, CROSS, SELF

SELECT s.name, c.course_name

FROM Students s

INNER JOIN Enrollments e ON s.student_id = e.student_id

INNER JOIN Courses c ON e.course_id = c.course_id;

The screenshot shows the phpMyAdmin interface with a query executed. The query is: `SELECT s.name, c.course_name FROM Students s INNER JOIN Enrollments e ON s.student_id = e.student_id INNER JOIN Courses c ON e.course_id = c.course_id;` The result table has two columns: 'name' and 'course_name'. The data is as follows:

name	course_name
chithra	DBMS
vinod	DBMS
deepa	DBMS
shree	Python
chandu	Python
chithra	Python
divya	AI
priya	AI
shree	AI
sony	ML
moulya	ML
divya	ML
deepa	Cloud
deeksha	Cloud
sony	Cloud

SELECT s.name, c.course_name

FROM Students s

LEFT JOIN Enrollments e ON s.student_id = e.student_id

LEFT JOIN Courses c ON e.course_id = c.course_id;

The screenshot shows the phpMyAdmin interface with a query executed. The query is: `SELECT s.name, c.course_name FROM Students s LEFT JOIN Enrollments e ON s.student_id = e.student_id LEFT JOIN Courses c ON e.course_id = c.course_id;` The result table has two columns: 'name' and 'course_name'. The data is as follows:

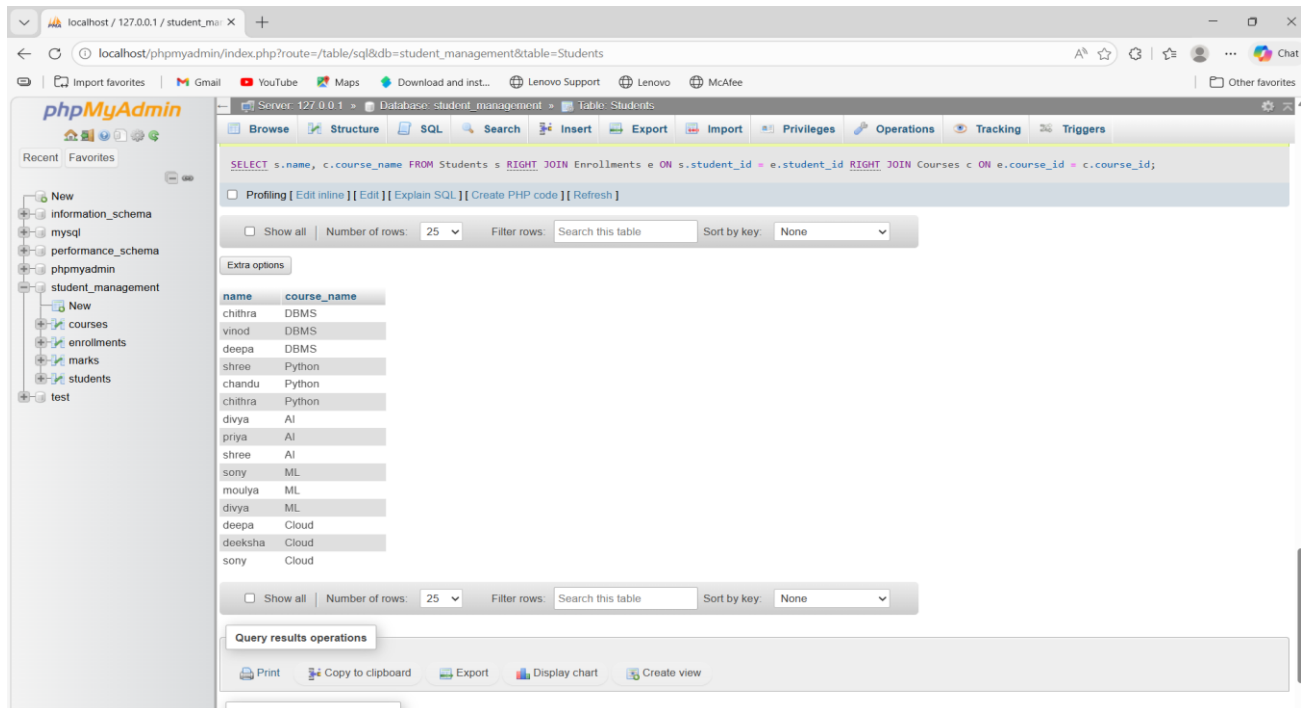
name	course_name
chithra	DBMS
chithra	Python
shree	Python
shree	AI
divya	AI
divya	ML
sony	ML
sony	Cloud
deepa	Cloud
deepa	DBMS
vinod	DBMS
chandu	Python
priya	AI
moulya	ML
deeksha	Cloud

```
SELECT s.name, c.course_name
```

```
FROM Students s
```

```
RIGHT JOIN Enrollments e ON s.student_id = e.student_id
```

```
RIGHT JOIN Courses c ON e.course_id = c.course_id;
```



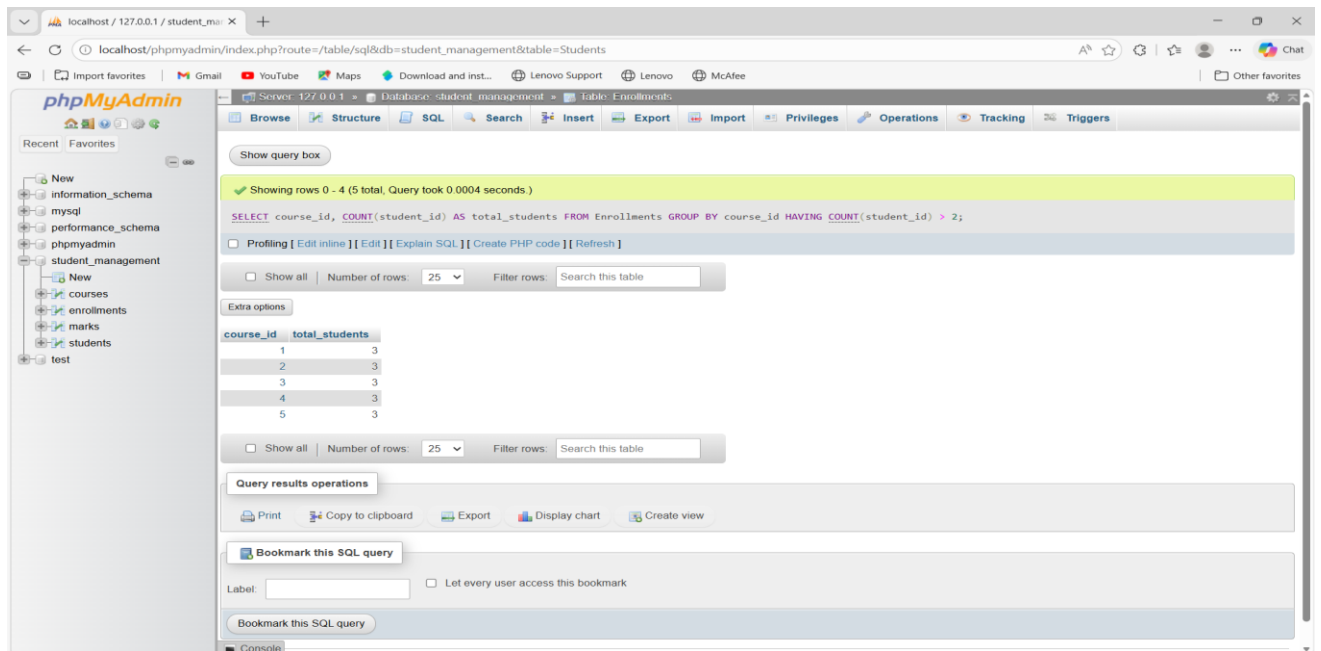
6 Grouping: GROUP BY, HAVING

```
SELECT course_id, COUNT(student_id) AS total_students
```

```
FROM Enrollments
```

```
GROUP BY course_id
```

```
HAVING COUNT(student_id) > 2;
```

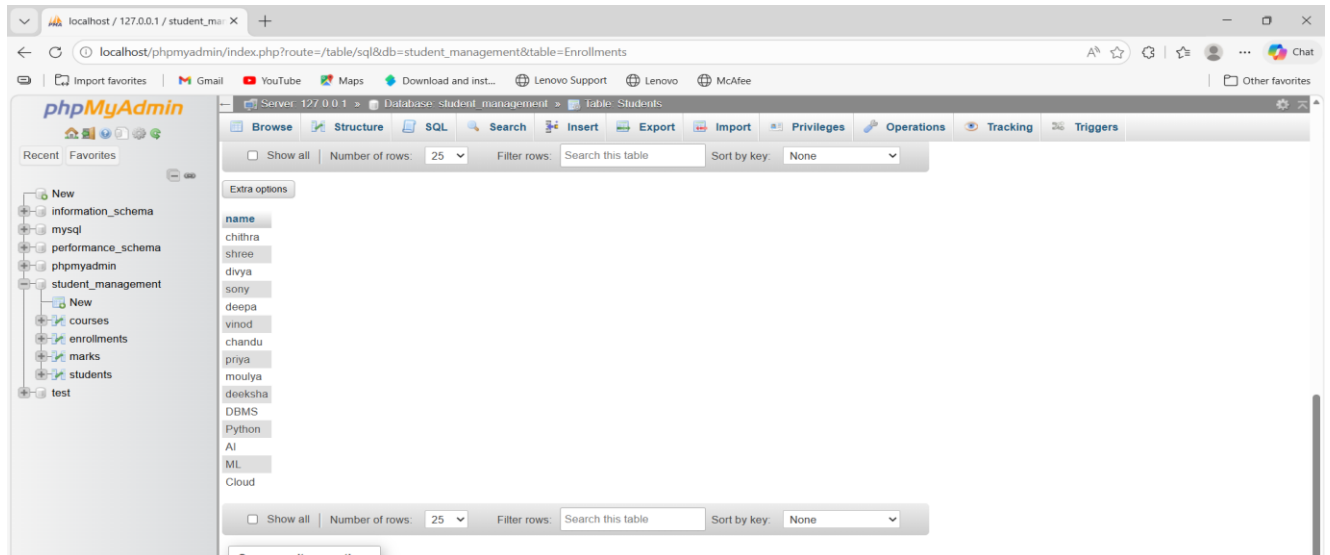


7 Advanced: UNION, EXISTS, CASE

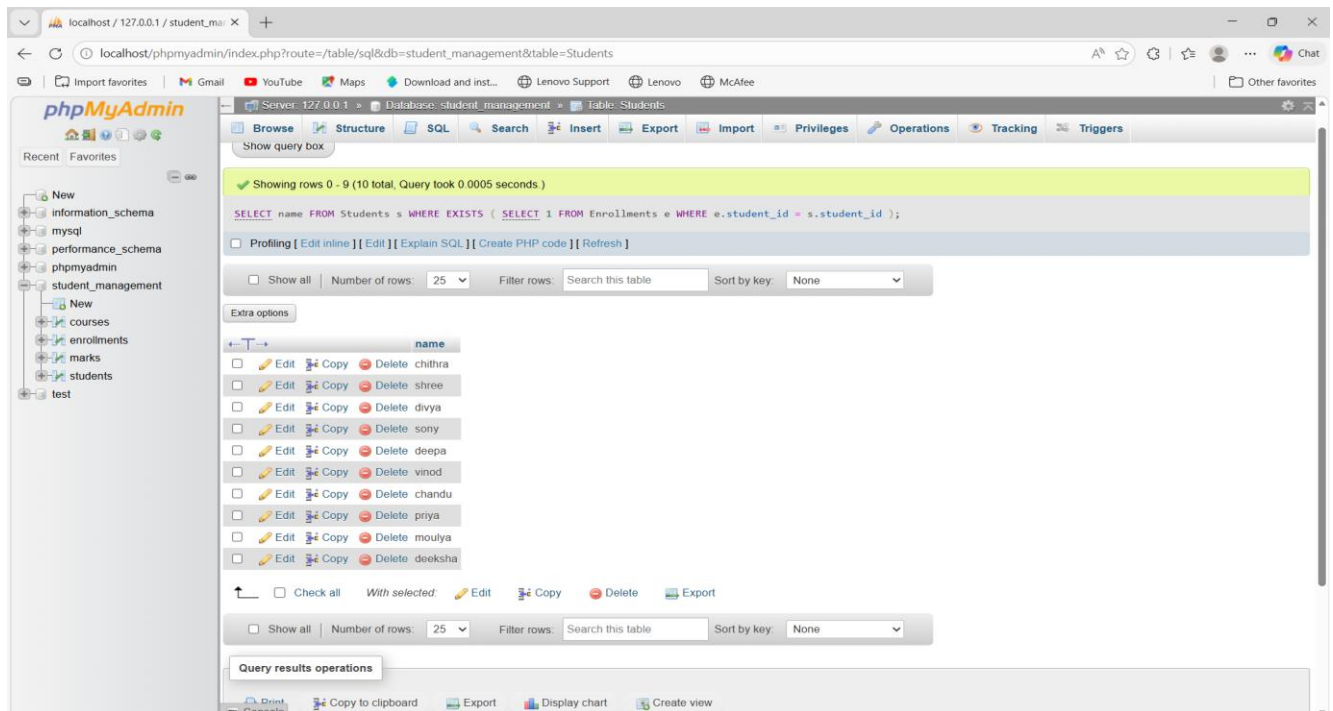
SELECT name FROM Students

UNION

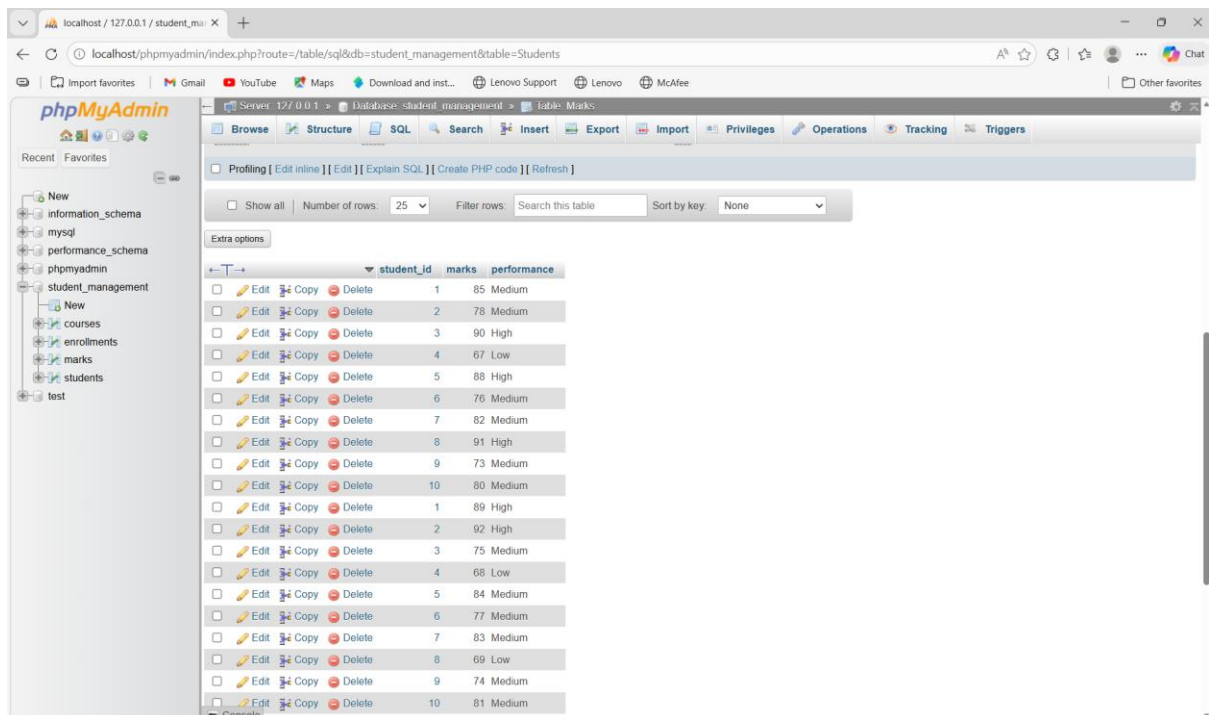
SELECT course_name FROM Courses;



```
SELECT name
FROM Students s
WHERE EXISTS (
SELECT 1 FROM Enrollments e WHERE e.student_id = s.student_id
);
```



SELECT student_id, marks,
CASE
WHEN marks > 85 THEN 'High'
WHEN marks BETWEEN 70 AND 85 THEN 'Medium'
ELSE 'Low'
END AS performance
FROM Marks;



Part 4: Data Modification

Perform UPDATE, DELETE, and INSERT SELECT operations

UPDATE Marks

SET marks = marks + 5

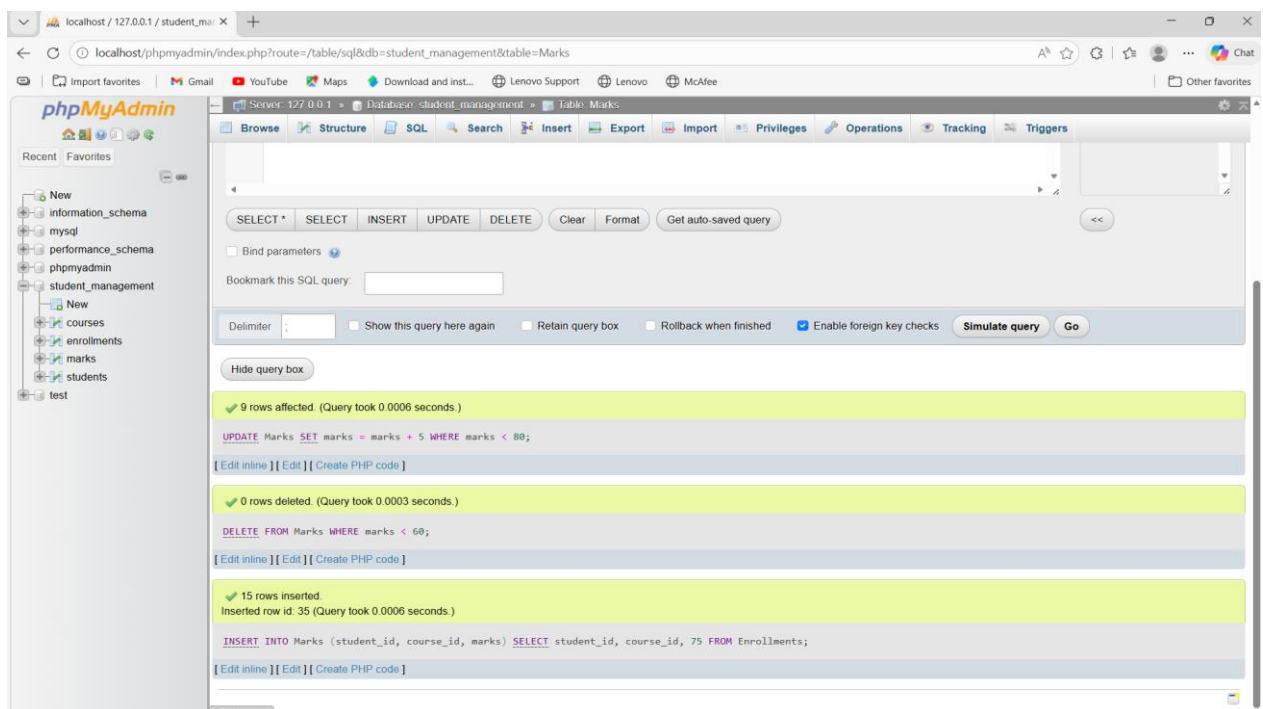
WHERE marks < 80;

DELETE FROM Marks

WHERE marks < 60;

INSERT INTO Marks (student_id, course_id, marks)

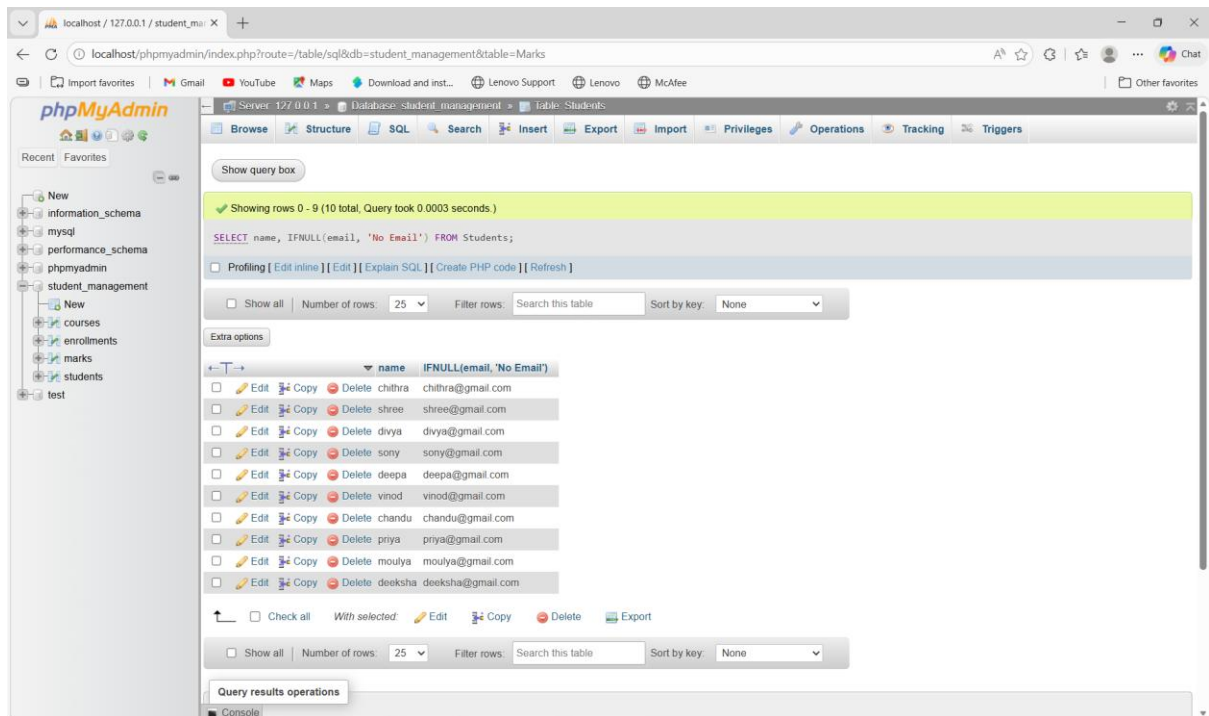
SELECT student_id, course_id, 75 FROM Enrollments;



Part 5: Advanced Features

1 Null handling functions (IFNULL, COALESCE)

SELECT name, IFNULL(email, 'No Email') FROM Students;



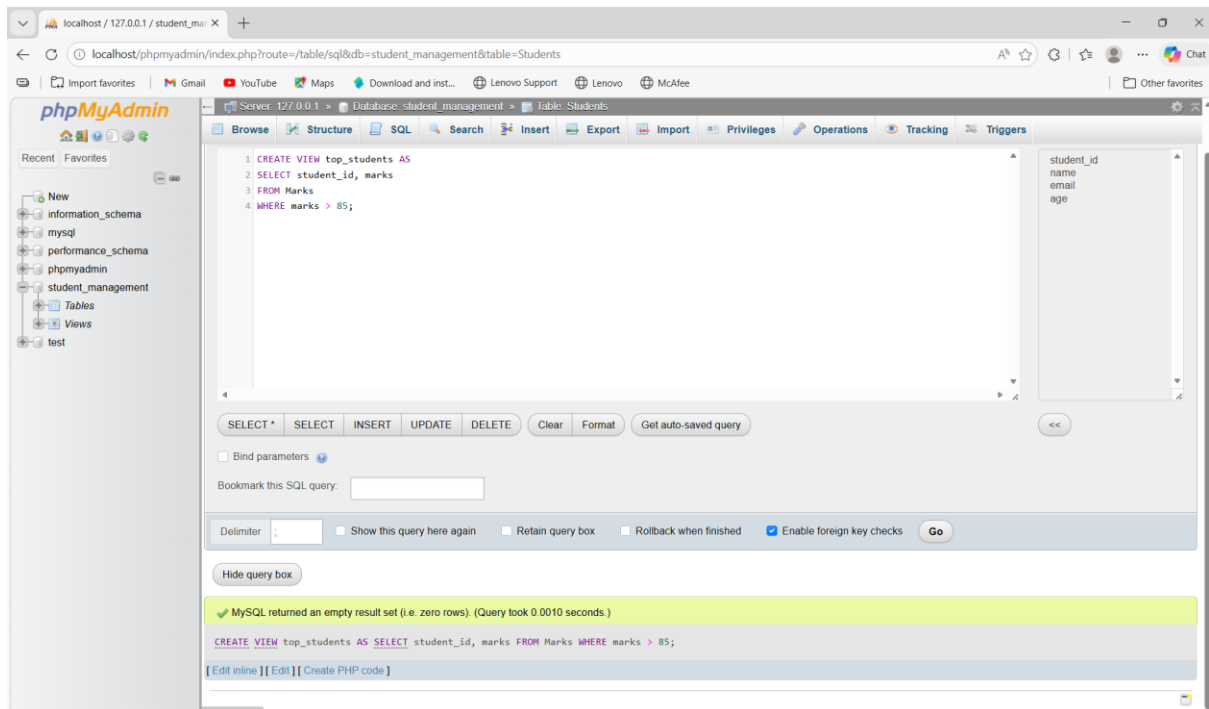
2 VIEWS CREATION

CREATE VIEW top_students AS

SELECT student_id, marks

FROM Marks

WHERE marks > 85;



3 Stored Procedures

DELIMITER //

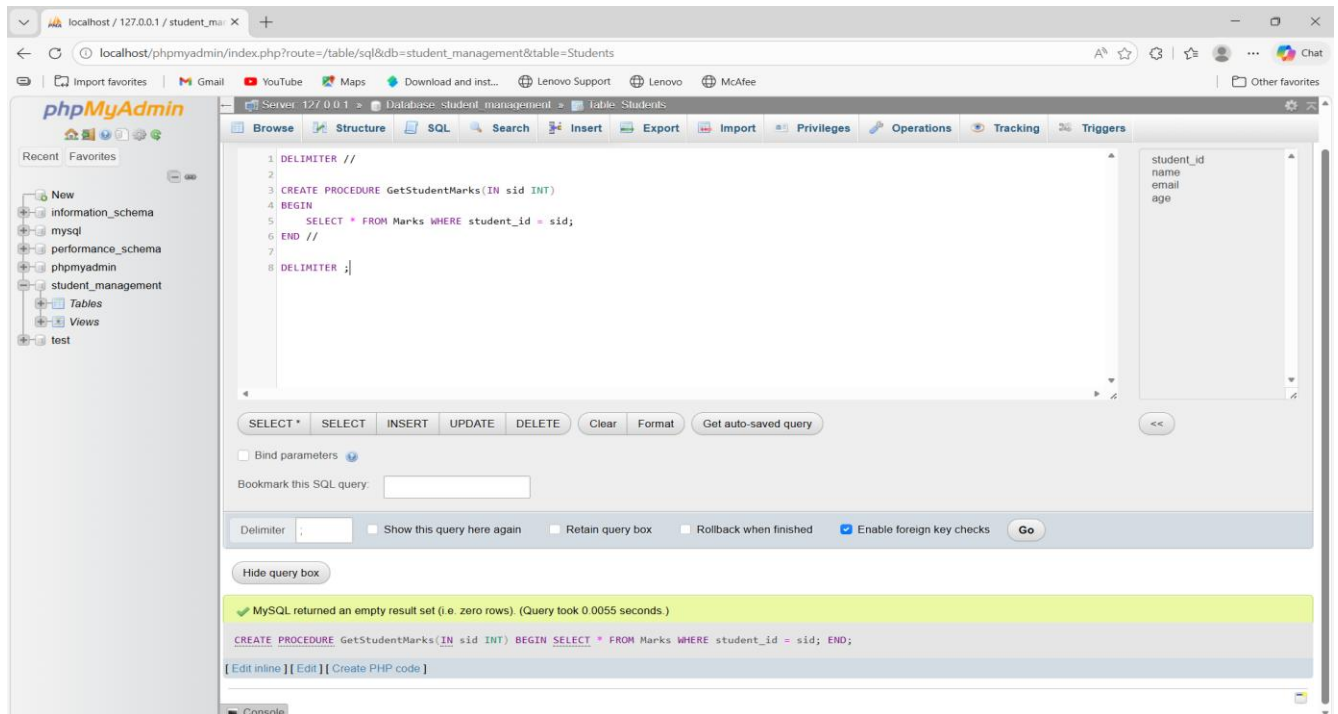
CREATE PROCEDURE GetStudentMarks(IN sid INT)

BEGIN

SELECT * FROM Marks WHERE student_id = sid;

END //

DELIMITER ;



4 Indexes and table alterations

CREATE INDEX idx_name ON Students(name);

